Python time Module: Zero to Professional

The time module lets you **work with time**, including:

- Getting current time
- Measuring execution time
- Pausing execution
- Formatting dates and times
- Converting between time formats

---

1. Importing the module

```
import time
```

Always import it at the top of your script.

---

2. Epoch Time (Timestamp)

- **Definition:** Number of seconds since **January 1, 1970, 00:00:00 UTC**.
- **Function:** time.time()

```
timestamp = time.time()
print(timestamp)  # e.g., 1731174000.123456
```

Pro Tip:

- Store timestamps in databases or logs for easy calculation of durations.

---

3. Pausing execution

- **Function:** time.sleep(seconds)

```
print("Start")
time.sleep(2)  # pauses program for 2 seconds
print("End after 2 seconds")
```

Use cases:

- Delay between requests (API calls)
- Animations
- Simulate long-running tasks

---

4. Structured Time

- time.localtime([seconds]) → Local time
- time.gmtime([seconds]) → UTC time
- Returns a **struct_time** object:

```
t = time.localtime()
print(t)
# time.struct_time(tm_year=2025, tm_mon=11, tm_mday=11, tm_hour=14, tm_min=0, tm_sec=30,
tm_wday=0, tm_yday=315, tm_isdst=0)
```

Attributes of struct_time:

- tm_year, tm_mon, tm_mday → year, month, day
- tm_hour, tm_min, tm_sec → hour, minute, second
- tm_wday → weekday (0=Monday)
- tm_yday → day of the year
- tm_isdst → daylight saving flag

---

5. Formatting Time

- time.strftime(format, t) → Convert struct_time to **readable string**

```
formatted_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
print(formatted_time)  # e.g., 2025-11-11 14:00:30
```

Common format codes:

Code	Meaning	Example
%Y	Year (4 digits)	2025
%m	Month (01-12)	11
%d	Day (01-31)	11
%H	Hour (24h)	14
%M	Minute	00
%S	Second	30
%a	Weekday short	Tue
%A	Weekday full	Tuesday
%b	Month short	Nov

Code	Meaning	Example
%B	Month full	November

- time.strptime(string, format) → Convert string to struct_time

```
time_str = "2025-11-11 14:00:30"
t = time.strptime(time_str, "%Y-%m-%d %H:%M:%S")
print(t.tm_year)  # 2025
```

---

6. Measuring Execution Time

- time.time() method:

```
start = time.time()
# some heavy code
for i in range(1000000):
    pass
end = time.time()
print("Elapsed:", end - start, "seconds")
```

- **Professional tip:** Use time.perf_counter() for **high-precision measurements**:

```
start = time.perf_counter()
# code
end = time.perf_counter()
print("High precision elapsed:", end - start)
```

---

7. Converting Between Time Formats

- time.mktime(t) → Convert struct_time to timestamp

```
t = time.localtime()
timestamp = time.mktime(t)
```

- time.asctime([t]) → Convert struct_time to readable string

```
t = time.localtime()
print(time.asctime(t))  # Tue Nov 11 14:00:30 2025
```

- time.ctime([seconds]) → Convert timestamp to string

```
print(time.ctime())  # Tue Nov 11 14:00:30 2025
```

---

8. Professional Tips

1. **Use datetime module** for more advanced date-time operations; time is more low-level.
2. **Avoid busy-wait loops**; use time.sleep() for delays.

3. **Timestamps** are excellent for performance measurement and logging.
4. **Always handle time zones** properly; time defaults to local time or UTC.

✅ Quick Reference Table

Task	Function
Current timestamp	time.time()
Pause program	time.sleep(seconds)
Local time	time.localtime()
UTC time	time.gmtime()
Format time	time.strftime(format, t)
Parse string → struct_time	time.strptime(string, format)
struct_time → timestamp	time.mktime(t)
struct_time → string	time.asctime(t)
timestamp → string	time.ctime(timestamp)
High precision timer	time.perf_counter()